

Neural Network Classification on MNIST Dataset

Neural Network Classification on MNIST Dataset

1. Introduction

This project focuses on implementing a Multi-Layer Perceptron (MLP) from scratch using Python and NumPy to classify handwritten digits from the MNIST dataset.

The dataset consists of 60,000 training images and 10,000 test images, where each digit (0-9) is represented as a grayscale 28×28 pixel image.

The goal is to:

- Implement the neural network manually (no ML libraries)
- Compare multiple hyperparameter configurations
- Evaluate accuracy, mean squared error (MSE), and timing
- Determine the optimal architecture and batch strategy

The experiment includes variations in:

- Activation functions → Case-1 vs Case-2
- Hidden layer sizes → 400 vs 800 neurons
- Learning rates → $\eta = 0.01$ vs 0.03
- Batch sizes → 20 vs 200

2. Methodology

2.1 Data Preparation

Each MNIST image (28×28) was flattened into a vector of length 784. Pixel values originally ranged from 0 to 255 and were normalized to [0,1] by dividing by 255.

The output labels were encoded using the predefined mapping rules:

- The output layer contains 10 neurons, representing digits 1,2,...,9,0 in that specific order.
- Desired output values are:
 - +1 for the neuron corresponding to the correct digit,
 - -1 for all other neurons.

This format follows the assignment requirement rather than softmax classification.

2.2 Network Architecture

A 2-layer neural network was implemented consisting of:

1. Input Layer → 784 neurons (+ bias)
2. Hidden Layer → $N = \{400, 800\}$ neurons (+ bias)
3. Output Layer → 10 neurons

Two activation configurations were tested:

1. Case 1: Hidden Layer Activation = $\tanh$, Output Activation = $\tanh$
2. Case 2: Hidden Layer Activation = ReLU, Output Activation = $\tanh$

Weights were initialized with small Gaussian noise to avoid signal saturation.

2.3 Training Procedure

The error function minimized during training is the mean squared error (MSE):

$$E(n) = 0.5 \sum_{i=1}^{10} (d_i(n) - o_i(n))^2$$

Training was performed using batch gradient descent with:

- Batch size $B = 20$
- Learning rates: $\eta = 0.01, 0.03$
- 60 epochs for each configuration

After evaluating all configurations, the best-performing one (based on test accuracy) was retrained using batch learning with batch sizes:

- $B = 20$
- $B = 200$

3. Experimental Results

A total of 8 configuration runs were completed from the combinations of activation case, hidden size, and learning rates. After training each model, the following metrics were recorded:

- Training accuracy
- Test accuracy
- Final loss (MSE)
- Number of misclassified samples per digit
- Average CPU time per epoch
- Total training duration
- Inference time per input sample

3.1 Summary Table of All Experiments

Case	Hidden	η	Train Acc	Test Acc	Train MSE	Test MSE	Avg Epoch Time
1	400	0.01	0.9892	0.9776	0.0440	0.0825	~0.75 s
1	400	0.03	0.9957	0.9819	0.0136	0.0696	~0.78 s
1	800	0.01	0.9872	0.9764	0.0519	0.0887	~1.77 s
1	800	0.03	0.9954	0.9826	0.0148	0.0686	~1.74 s
2	400	0.01	0.9946	0.9831	0.0225	0.0668	~0.77 s
2	400	0.03	0.9973	0.9839	0.0076	0.0631	~0.76 s
2	800	0.01	0.9953	0.9832	0.0197	0.0644	~1.74 s
2	800	0.03	0.9978	0.9842	0.0063	0.0591	~1.70 s

Table 1: Output Summary

3.2 Best Performing Configuration

The best configuration based on highest test accuracy was:

- Case 2: ReLU (hidden) + $\tanh$ (output), 800 hidden neurons, $\eta = 0.03$

This model achieved:

- Training accuracy: 0.9978
- Test accuracy: 0.9842
- Average inference time: 2.61×10^{-6} seconds per digit

3.3 Batch Size Comparison (Best Model Only)

Batch Size	Train Accuracy	Test Accuracy	Train MSE	Test MSE	Training Time
20	0.9977	0.9849	0.0063	0.0593	~101 s
200	0.9843	0.9763	0.0679	0.0884	~24 s

Table 2: Best Model Batch Size Comparison Summary

Observation:

- B=20 gives the best accuracy, but slower training.
- B=200 trains 4× faster, but accuracy drops.

This reflects the classic trade-off between stability (small batch) and speed (large batch).

4. Discussion

- Case 2 (ReLU + tanh) consistently outperformed Case 1.
- Increasing hidden neurons from 400 → 800 improved test accuracy, especially in Case 2.
- A higher learning rate ($\eta = 0.03$) produced faster convergence and lower final error without instability.
- Misclassification analysis suggests most confusion occurred between digits with similar shapes (e.g., 5 vs. 3, 8 vs. 9).
- Smaller batch (B=20) produced smoother, more reliable convergence and better final generalization.

5. Conclusion

This assignment demonstrates that a properly tuned MLP can achieve strong performance on MNIST without advanced deep learning frameworks. The best-performing configuration achieved:

98.49% accuracy on the test set using Case 2, 800 hidden neurons, $\eta = 0.03$, batch size 20.

The project highlights the impact of architectural and optimization hyperparameters such as activation function, network width, learning rate, and batch size on classification performance.

Appendix A - Figures

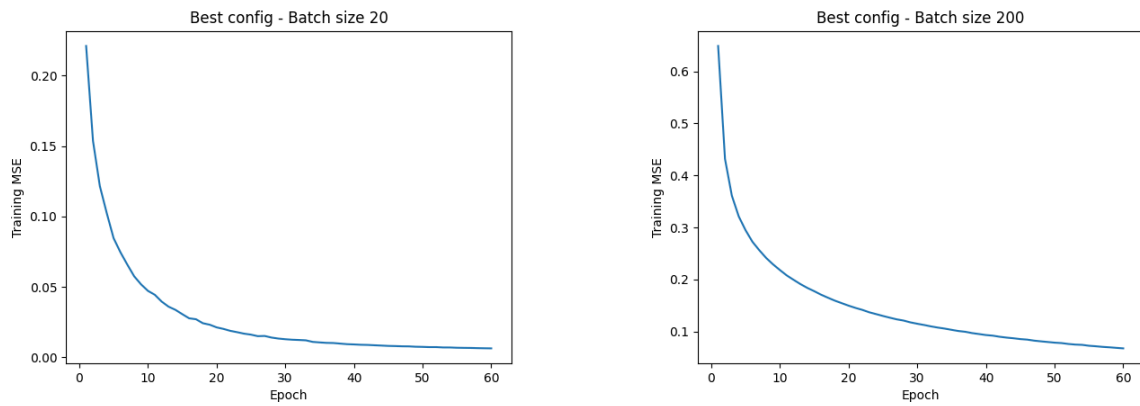


Figure 1,2: Best Configuration with Batch Size 20, 200 respectively

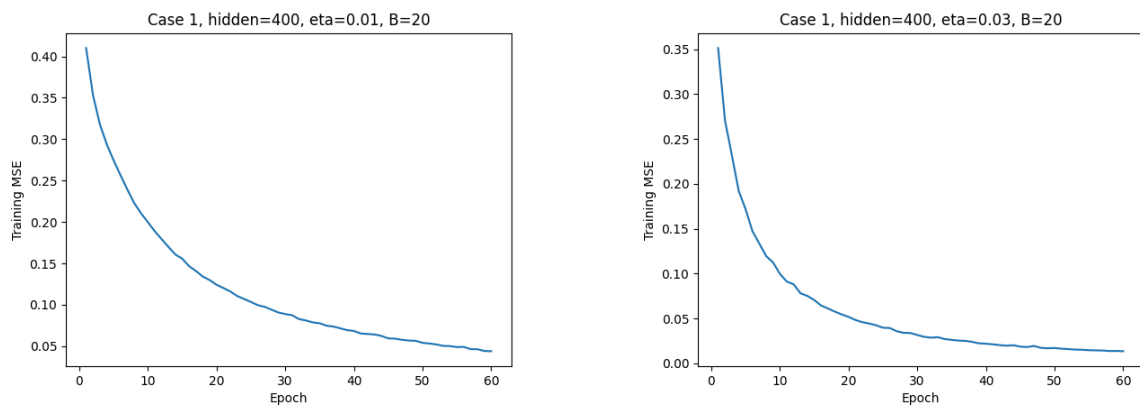


Figure 3,4: Case 1, Batch size = 20, Hidden=400, eta = 0.01, 0.03 respectively

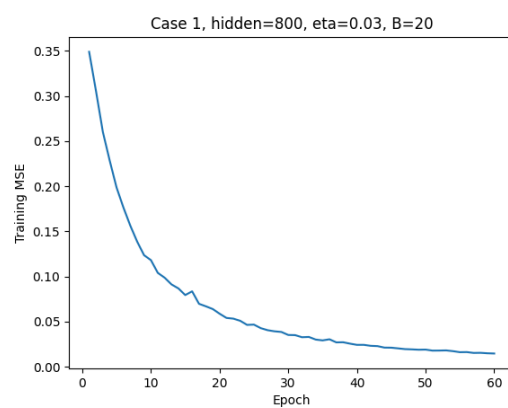
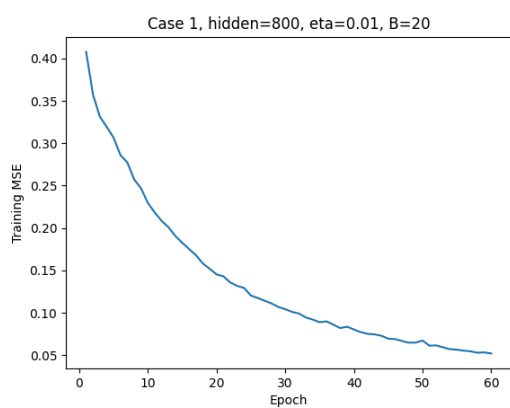


Figure 5,6: Case 1, Batch size = 20, Hidden=800, eta = 0.01 , 0.03 respectively

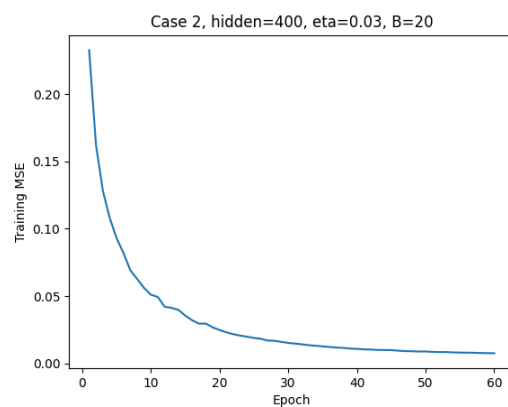
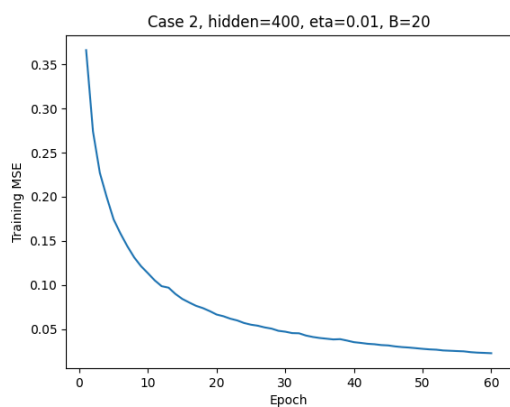


Figure 7,8: Case 2, Batch size = 20, Hidden=400, eta = 0.01 , 0.03 respectively

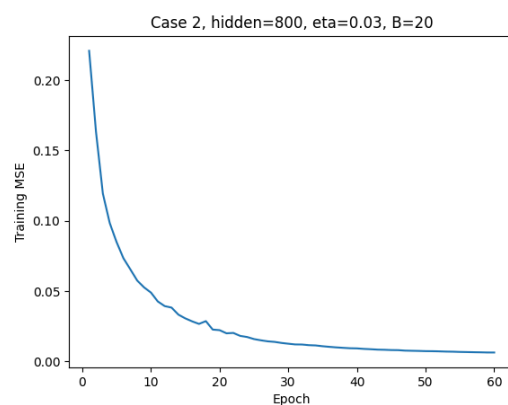
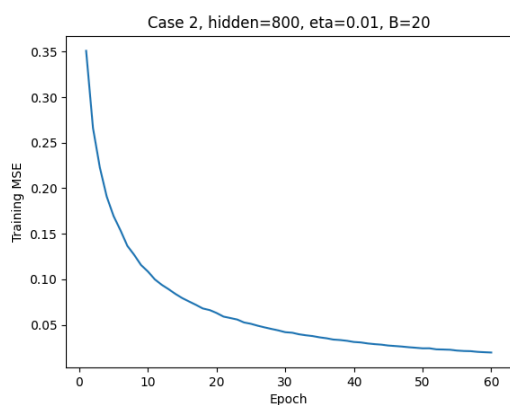


Figure 9,10: Case 2, Batch size = 20, Hidden=800, eta = 0.01 , 0.03 respectively

Appendix B - Code

```
import numpy as np
import struct
import time
import matplotlib

# NumPy uyarılarını sustur (overflow vs.)
np.seterr(all='ignore')

# ----- MNIST LOADING HELPERS ----- #

def load_mnist_images(filename):
    """Reads MNIST image file and returns (N, 784) float32 array in [0,1]."""
    with open(filename, 'rb') as f:
        magic, num, rows, cols = struct.unpack('>IIII', f.read(16))
        data = np.frombuffer(f.read(), dtype=np.uint8)
        data = data.reshape(num, rows * cols).astype(np.float32) / 255.0
    return data

def load_mnist_labels(filename):
    """Reads MNIST label file and returns (N,) uint8 array with digits 0–9."""
    with open(filename, 'rb') as f:
        magic, num = struct.unpack('>II', f.read(8))
        labels = np.frombuffer(f.read(), dtype=np.uint8)
    return labels

def labels_to_targets(labels):
    """
    Converts digit labels (0–9) to desired output vectors of length 10.

    Output neuron order: 1,2,3,4,5,6,7,8,9,0
    For digit i: only corresponding neuron is +1, others are -1.
    """
    N = labels.shape[0]
    targets = -np.ones((N, 10), dtype=np.float32)
    # digit → neuron index: 1→0, 2→1, ..., 9→8, 0→9
    indices = np.where(labels == 0, 9, labels - 1)
```

```

targets[np.arange(N), indices] = 1.0
return targets

# ----- ACTIVATION FUNCTIONS ----- #

def tanh(x):
    return np.tanh(x)

def tanh_deriv_from_output(y):
    # derivative of tanh wrt input, expressed via output  $y = \tanh(v)$ 
    return 1.0 - y ** 2

def relu(x):
    return np.maximum(0.0, x)

def relu_deriv_from_input(x):
    return (x > 0).astype(np.float32)

# ----- MLP IMPLEMENTATION ----- #

class MLP:
    """
    2-layer MLP: input → hidden → output
    Bias is handled by adding an extra input of 1 at each layer.
    """

    def __init__(self, input_dim, hidden_dim, output_dim,
                  activation_hidden='tanh', activation_output='tanh',
                  eta=0.01, seed=None):

        if seed is not None:
            np.random.seed(seed)

        self.input_dim = input_dim
        self.hidden_dim = hidden_dim
        self.output_dim = output_dim
        self.eta = eta

```

```

# choose activations
if activation_hidden == 'tanh':
    self.act_h = tanh
    self.d_act_h_input = None    # use output-based deriv
elif activation_hidden == 'relu':
    self.act_h = relu
    self.d_act_h_input = relu_deriv_from_input
else:
    raise ValueError("Unsupported hidden activation.")

if activation_output == 'tanh':
    self.act_o = tanh
    self.d_act_o_output = tanh_deriv_from_output
else:
    raise ValueError("Unsupported output activation.")

# Small Gaussian initialization to avoid overflow
self.W1 = (0.01 * np.random.randn(input_dim + 1, hidden_dim)).astype(np.float32)
self.W2 = (0.01 * np.random.randn(hidden_dim + 1, output_dim)).astype(np.float32)

# ---- forward ---- #

def _forward(self, X):
    """
    Forward pass.
    X: (batch, input_dim)
    returns: Xb, Z1, H, Hb, Z2, O
    """
    batch_size = X.shape[0]

    # add bias to input
    Xb = np.concatenate([X, np.ones((batch_size, 1), dtype=np.float32)],
                        axis=1)
    Z1 = Xb @ self.W1
    Z1 = np.clip(Z1, -50, 50)
    H = self.act_h(Z1)

    # add bias to hidden activations
    Hb = np.concatenate([H, np.ones((batch_size, 1), dtype=np.float32)],
                        axis=1)
    Z2 = Hb @ self.W2

```



```

Z2 = np.clip(Z2, -50, 50)
O = self.act_o(Z2)

return Xb, Z1, H, Hb, Z2, O

def predict(self, X):
    """Returns network output for given inputs X."""
    _, _, _, _, _, O = self._forward(X)
    return O

# ---- training (batch) ---- #

def _update_batch(self, X_batch, D_batch):
    """
    Performs one gradient step using a batch.
    X_batch: (B, input_dim)
    D_batch: (B, output_dim)
    """
    Xb, Z1, H, Hb, Z2, O = self._forward(X_batch)
    B = X_batch.shape[0]

    # output layer delta
    dE_dO = (O - D_batch) # (B, out)
    delta_o = dE_dO * self.d_act_o_output(O) # (B, out)

    # hidden layer delta (exclude bias column from W2)
    d_hidden = (delta_o @ self.W2.T)[:, :-1] # (B, hidden)
    if self.d_act_h_input is None:
        # tanh: derivative via output H
        d_act_h = tanh_deriv_from_output(H)
    else:
        # ReLU: derivative via input Z1
        d_act_h = self.d_act_h_input(Z1)

    delta_h = d_hidden * d_act_h # (B, hidden)

    # gradients
    grad_W2 = Hb.T @ delta_o / B
    grad_W1 = Xb.T @ delta_h / B

    # gradient descent step
    self.W2 -= self.eta * grad_W2
    self.W1 -= self.eta * grad_W1

```

```

def train_batch(self, X, D, batch_size=20, epochs=60, verbose=True):
    """
    Batch learning with given batch size.
    Returns (epoch_times, loss_history).
    """
    N = X.shape[0]
    epoch_times = []
    loss_history = []

    for epoch in range(epochs):
        t0 = time.time()

        idx = np.random.permutation(N)
        X_sh = X[idx]
        D_sh = D[idx]

        for start in range(0, N, batch_size):
            end = min(start + batch_size, N)
            xb = X_sh[start:end]
            db = D_sh[start:end]
            self._update_batch(xb, db)

        t1 = time.time()
        epoch_times.append(t1 - t0)

        mse = self.compute_mse(X, D)
        loss_history.append(mse)

        if verbose:
            print(f"[batch B={batch_size}] epoch {epoch + 1}/{epochs}, "
                  f"MSE={mse:.4f}, time={epoch_times[-1]:.3f}s")

    return epoch_times, loss_history

# ---- metrics ---- #

def compute_mse(self, X, D):
    O = self.predict(X)
    return 0.5 * np.mean(np.sum((D - O) ** 2, axis=1))

def evaluate_classification(self, X, labels):
    """

```

```

Returns: accuracy, mse, misclassified_per_digit (dict digit→count)
"""
O = self.predict(X)
# index 0..9 → digits 1..9,0
pred_idx = np.argmax(O, axis=1)
pred_digits = np.where(pred_idx == 9, 0, pred_idx + 1)

correct = (pred_digits == labels)
accuracy = correct.mean()

misclassified_per_digit = {}
for d in range(10):
    mask = (labels == d)
    mis = np.logical_and(mask, ~correct).sum()
    misclassified_per_digit[d] = int(mis)

D = labels_to_targets(labels)
mse = self.compute_mse(X, D)

return accuracy, mse, misclassified_per_digit

def inference_time_per_sample(model, X, repeats=5):
    """Roughly measures average time to classify a single pattern."""
    N = X.shape[0]
    t0 = time.time()
    for _ in range(repeats):
        model.predict(X)
    t1 = time.time()
    return (t1 - t0) / (repeats * N)

# ----- MAIN EXPERIMENT ----- #

def main():
    # 1) Load data
    train_images_file = "train-images-idx3-ubyte"
    train_labels_file = "train-labels-idx1-ubyte"
    test_images_file = "t10k-images-idx3-ubyte"
    test_labels_file = "t10k-labels-idx1-ubyte"

    print("Loading MNIST files...")
    X_train = load_mnist_images(train_images_file)

```

```

y_train = load_mnist_labels(train_labels_file)
X_test = load_mnist_images(test_images_file)
y_test = load_mnist_labels(test_labels_file)

print(f"Train samples: {X_train.shape[0]}, dim: {X_train.shape[1]}")
print(f"Test samples : {X_test.shape[0]}, dim: {X_test.shape[1]}")

input_dim = X_train.shape[1] # should be 784
output_dim = 10

hidden_list = [400, 800]
etas = [0.01, 0.03]
cases = [1, 2] # 1: tanh-tanh, 2: ReLU-tanh
epochs_batch = 60
batch_main = 20

D_train = labels_to_targets(y_train)

results = []
loss_curves = {} # (case, hidden, eta) → loss_history

# ----- TRAIN ALL CONFIGS WITH BATCH (B=20) ----- #
for case in cases:
    for N_hid in hidden_list:
        for eta in etas:
            if case == 1:
                act_h, act_o = 'tanh', 'tanh'
            else:
                act_h, act_o = 'relu', 'tanh'

            print("\n=== CONFIG: case={}, hidden={}, eta={} (B=20) ==="
                  .format(case, N_hid, eta))

            model = MLP(input_dim, N_hid, output_dim,
                        activation_hidden=act_h,
                        activation_output=act_o,
                        eta=eta)

            epoch_times, loss_hist = model.train_batch(
                X_train, D_train,
                batch_size=batch_main,
                epochs=epochs_batch,
                verbose=True

```

```

)

avg_epoch_time = float(np.mean(epoch_times))
total_train_time = float(np.sum(epoch_times))

# loss eğrilerini kaydet
loss_curves[(case, N_hid, eta)] = loss_hist

# Train metrics
train_acc, train_mse, train_mis = \
    model.evaluate_classification(X_train, y_train)

# Test metrics
test_acc, test_mse, test_mis = \
    model.evaluate_classification(X_test, y_test)

inf_time = inference_time_per_sample(model, X_test)

print(f"Train acc: {train_acc:.4f}, "
      f"Test acc: {test_acc:.4f}")
print(f"Train MSE: {train_mse:.4f}, "
      f"Test MSE: {test_mse:.4f}")
print(f"Avg epoch time: {avg_epoch_time:.4f}s, "
      f"Total train time: {total_train_time:.2f}s")
print(f"Inference time per sample: {inf_time:.6e}s")
print("Misclassified per digit (test):", test_mis)

results.append({
    "case": case,
    "hidden": N_hid,
    "eta": eta,
    "train_acc": train_acc,
    "train_mse": train_mse,
    "test_acc": test_acc,
    "test_mse": test_mse,
    "train_mis": train_mis,
    "test_mis": test_mis,
    "avg_epoch_time": avg_epoch_time,
    "total_train_time": total_train_time,
    "inf_time_per_sample": inf_time,
})

# ----- PLOT TRAINING LOSS CURVES FOR ALL CONFIGS (B=20) ----- #

```

```

for key, losses in loss_curves.items():
    case, N_hid, eta = key
    epochs = np.arange(1, len(losses) + 1)

    plt.figure()
    plt.plot(epochs, losses)
    plt.xlabel("Epoch")
    plt.ylabel("Training MSE")
    plt.title(f"Case {case}, hidden={N_hid}, eta={eta}, B=20")
    import os
    out_dir = "results_3"
    os.makedirs(out_dir, exist_ok=True)
    filename = f"case{case}_hidden{N_hid}_eta{eta}_B20.png"
    plt.savefig(os.path.join(out_dir, filename))
    plt.grid(True)

# plt.show()

# ----- SELECT BEST CONFIG BY TEST ACCURACY ----- #
best = max(results, key=lambda r: r["test_acc"])
print("\n=== BEST CONFIGURATION (BY TEST ACC) ===")
print("case = {}, hidden = {}, eta = {}, test_acc = {:.4f}"
      .format(best["case"], best["hidden"],
              best["eta"], best["test_acc"]))

# ----- BATCH LEARNING FOR BEST CONFIG WITH B=20 AND B=200 -----
-- #
B_values = [20, 200]

for B in B_values:
    print("\n=== BATCH LEARNING: B={}, case={}, hidden={}, eta={} ==="
          .format(B, best["case"], best["hidden"], best["eta"]))

    if best["case"] == 1:
        act_h, act_o = 'tanh', 'tanh'
    else:
        act_h, act_o = 'relu', 'tanh'

    model = MLP(input_dim, best["hidden"], output_dim,
                 activation_hidden=act_h,
                 activation_output=act_o,
                 eta=best["eta"])

```

```

epoch_times, loss_hist_B = model.train_batch(
    X_train, D_train,
    batch_size=B,
    epochs=epochs_batch,
    verbose=True
)

avg_epoch_time = float(np.mean(epoch_times))
total_train_time = float(np.sum(epoch_times))

train_acc, train_mse, train_mis = \
    model.evaluate_classification(X_train, y_train)
test_acc, test_mse, test_mis = \
    model.evaluate_classification(X_test, y_test)
inf_time = inference_time_per_sample(model, X_test)

print(f"[B={B}] Train acc: {train_acc:.4f}, "
      f"Test acc: {test_acc:.4f}")
print(f"[B={B}] Train MSE: {train_mse:.4f}, "
      f"Test MSE: {test_mse:.4f}")
print(f"[B={B}] Avg epoch time: {avg_epoch_time:.4f}s, "
      f"Total train time: {total_train_time:.2f}s")
print(f"[B={B}] Inference time per sample: {inf_time:.6e}s")
print(f"[B={B}] Misclassified per digit (test): {test_mis}")

# B için loss grafiği çiz
epochs_arr = np.arange(1, len(loss_hist_B) + 1)
plt.figure()
plt.plot(epochs_arr, loss_hist_B)
plt.xlabel("Epoch")
plt.ylabel("Training MSE")
plt.title(f"Best config - Batch size {B}")
import os
out_dir = "results_3"
os.makedirs(out_dir, exist_ok=True)
filename = f"bestconfig_batch{B}.png"
plt.savefig(os.path.join(out_dir, filename))
plt.grid(True)

# plt.show()

```

```
if __name__ == "__main__":  
    main()
```


Appendix C - Terminal Output

```
(base) handeeryilmaz@Hande-MacBook-Air 443_proje3 % python3 [Untitled-2.py]
(http://untitled-2.py/)
Loading MNIST files...
Train samples: 60000, dim: 784
Test samples : 10000, dim: 784

=== CONFIG: case=1, hidden=400, eta=0.01 (B=20) ===
[batch B=20] epoch 1/60, MSE=0.4099, time=0.619s
[batch B=20] epoch 2/60, MSE=0.3532, time=0.771s
[batch B=20] epoch 3/60, MSE=0.3180, time=0.784s
[batch B=20] epoch 4/60, MSE=0.2939, time=0.726s
[batch B=20] epoch 5/60, MSE=0.2745, time=0.828s
[batch B=20] epoch 6/60, MSE=0.2566, time=0.763s
[batch B=20] epoch 7/60, MSE=0.2393, time=0.761s
[batch B=20] epoch 8/60, MSE=0.2229, time=0.776s
[batch B=20] epoch 9/60, MSE=0.2105, time=0.684s
[batch B=20] epoch 10/60, MSE=0.1997, time=0.772s
[batch B=20] epoch 11/60, MSE=0.1888, time=0.807s
[batch B=20] epoch 12/60, MSE=0.1793, time=0.734s
[batch B=20] epoch 13/60, MSE=0.1699, time=0.746s
[batch B=20] epoch 14/60, MSE=0.1608, time=0.667s
[batch B=20] epoch 15/60, MSE=0.1557, time=0.783s
[batch B=20] epoch 16/60, MSE=0.1467, time=0.761s
[batch B=20] epoch 17/60, MSE=0.1408, time=0.754s
[batch B=20] epoch 18/60, MSE=0.1341, time=0.666s
[batch B=20] epoch 19/60, MSE=0.1299, time=0.775s
[batch B=20] epoch 20/60, MSE=0.1242, time=0.709s
[batch B=20] epoch 21/60, MSE=0.1203, time=0.704s
[batch B=20] epoch 22/60, MSE=0.1161, time=0.815s
[batch B=20] epoch 23/60, MSE=0.1107, time=0.661s
[batch B=20] epoch 24/60, MSE=0.1071, time=0.765s
[batch B=20] epoch 25/60, MSE=0.1035, time=0.745s
[batch B=20] epoch 26/60, MSE=0.0996, time=0.807s
[batch B=20] epoch 27/60, MSE=0.0976, time=0.783s
[batch B=20] epoch 28/60, MSE=0.0942, time=0.719s
[batch B=20] epoch 29/60, MSE=0.0908, time=0.779s
[batch B=20] epoch 30/60, MSE=0.0889, time=0.737s
[batch B=20] epoch 31/60, MSE=0.0874, time=0.818s
[batch B=20] epoch 32/60, MSE=0.0829, time=0.735s
```

```

[batch B=20] epoch 33/60, MSE=0.0811, time=0.774s
[batch B=20] epoch 34/60, MSE=0.0788, time=0.775s
[batch B=20] epoch 35/60, MSE=0.0778, time=0.709s
[batch B=20] epoch 36/60, MSE=0.0748, time=0.709s
[batch B=20] epoch 37/60, MSE=0.0738, time=0.728s
[batch B=20] epoch 38/60, MSE=0.0717, time=0.770s
[batch B=20] epoch 39/60, MSE=0.0695, time=0.701s
[batch B=20] epoch 40/60, MSE=0.0685, time=0.817s
[batch B=20] epoch 41/60, MSE=0.0654, time=0.720s
[batch B=20] epoch 42/60, MSE=0.0649, time=0.820s
[batch B=20] epoch 43/60, MSE=0.0642, time=0.748s
[batch B=20] epoch 44/60, MSE=0.0625, time=0.794s
[batch B=20] epoch 45/60, MSE=0.0595, time=0.757s
[batch B=20] epoch 46/60, MSE=0.0592, time=0.774s
[batch B=20] epoch 47/60, MSE=0.0578, time=0.658s
[batch B=20] epoch 48/60, MSE=0.0569, time=0.727s
[batch B=20] epoch 49/60, MSE=0.0565, time=0.775s
[batch B=20] epoch 50/60, MSE=0.0541, time=0.815s
[batch B=20] epoch 51/60, MSE=0.0533, time=0.770s
[batch B=20] epoch 52/60, MSE=0.0522, time=0.768s
[batch B=20] epoch 53/60, MSE=0.0505, time=0.716s
[batch B=20] epoch 54/60, MSE=0.0502, time=0.755s
[batch B=20] epoch 55/60, MSE=0.0490, time=0.721s
[batch B=20] epoch 56/60, MSE=0.0492, time=0.751s
[batch B=20] epoch 57/60, MSE=0.0465, time=0.772s
[batch B=20] epoch 58/60, MSE=0.0464, time=0.811s
[batch B=20] epoch 59/60, MSE=0.0443, time=0.722s
[batch B=20] epoch 60/60, MSE=0.0440, time=0.704s
Train acc: 0.9892, Test acc: 0.9776
Train MSE: 0.0440, Test MSE: 0.0825
Avg epoch time: 0.7503s, Total train time: 45.02s
Inference time per sample: 1.570144e-06s
Misclassified per digit (test): {0: 8, 1: 13, 2: 24, 3: 16, 4: 20, 5: 33, 6: 17, 7: 33, 8: 26,
9: 34}

```

```

=== CONFIG: case=1, hidden=400, eta=0.03 (B=20) ===
[batch B=20] epoch 1/60, MSE=0.3513, time=0.743s
[batch B=20] epoch 2/60, MSE=0.2707, time=0.785s
[batch B=20] epoch 3/60, MSE=0.2321, time=0.801s
[batch B=20] epoch 4/60, MSE=0.1922, time=0.781s
[batch B=20] epoch 5/60, MSE=0.1718, time=0.820s
[batch B=20] epoch 6/60, MSE=0.1472, time=0.753s

```

[batch B=20] epoch 7/60, MSE=0.1336, time=0.787s
[batch B=20] epoch 8/60, MSE=0.1195, time=0.806s
[batch B=20] epoch 9/60, MSE=0.1125, time=0.811s
[batch B=20] epoch 10/60, MSE=0.0998, time=0.796s
[batch B=20] epoch 11/60, MSE=0.0912, time=0.750s
[batch B=20] epoch 12/60, MSE=0.0881, time=0.817s
[batch B=20] epoch 13/60, MSE=0.0781, time=0.721s
[batch B=20] epoch 14/60, MSE=0.0752, time=0.795s
[batch B=20] epoch 15/60, MSE=0.0706, time=0.769s
[batch B=20] epoch 16/60, MSE=0.0645, time=0.801s
[batch B=20] epoch 17/60, MSE=0.0612, time=0.709s
[batch B=20] epoch 18/60, MSE=0.0577, time=0.790s
[batch B=20] epoch 19/60, MSE=0.0546, time=0.726s
[batch B=20] epoch 20/60, MSE=0.0518, time=0.819s
[batch B=20] epoch 21/60, MSE=0.0484, time=0.803s
[batch B=20] epoch 22/60, MSE=0.0459, time=0.794s
[batch B=20] epoch 23/60, MSE=0.0444, time=0.765s
[batch B=20] epoch 24/60, MSE=0.0425, time=0.726s
[batch B=20] epoch 25/60, MSE=0.0398, time=0.718s
[batch B=20] epoch 26/60, MSE=0.0394, time=0.772s
[batch B=20] epoch 27/60, MSE=0.0359, time=0.797s
[batch B=20] epoch 28/60, MSE=0.0342, time=0.703s
[batch B=20] epoch 29/60, MSE=0.0339, time=0.791s
[batch B=20] epoch 30/60, MSE=0.0317, time=0.797s
[batch B=20] epoch 31/60, MSE=0.0297, time=0.790s
[batch B=20] epoch 32/60, MSE=0.0287, time=0.818s
[batch B=20] epoch 33/60, MSE=0.0293, time=0.818s
[batch B=20] epoch 34/60, MSE=0.0272, time=0.809s
[batch B=20] epoch 35/60, MSE=0.0262, time=0.788s
[batch B=20] epoch 36/60, MSE=0.0255, time=0.780s
[batch B=20] epoch 37/60, MSE=0.0251, time=0.796s
[batch B=20] epoch 38/60, MSE=0.0240, time=0.776s
[batch B=20] epoch 39/60, MSE=0.0224, time=0.801s
[batch B=20] epoch 40/60, MSE=0.0219, time=0.813s
[batch B=20] epoch 41/60, MSE=0.0213, time=0.782s
[batch B=20] epoch 42/60, MSE=0.0204, time=0.810s
[batch B=20] epoch 43/60, MSE=0.0197, time=0.788s
[batch B=20] epoch 44/60, MSE=0.0203, time=0.784s
[batch B=20] epoch 45/60, MSE=0.0187, time=0.733s
[batch B=20] epoch 46/60, MSE=0.0182, time=0.730s
[batch B=20] epoch 47/60, MSE=0.0195, time=0.766s
[batch B=20] epoch 48/60, MSE=0.0174, time=0.803s
[batch B=20] epoch 49/60, MSE=0.0169, time=0.791s

```
[batch B=20] epoch 50/60, MSE=0.0172, time=0.794s
[batch B=20] epoch 51/60, MSE=0.0164, time=0.774s
[batch B=20] epoch 52/60, MSE=0.0160, time=0.818s
[batch B=20] epoch 53/60, MSE=0.0154, time=0.739s
[batch B=20] epoch 54/60, MSE=0.0152, time=0.799s
[batch B=20] epoch 55/60, MSE=0.0148, time=0.811s
[batch B=20] epoch 56/60, MSE=0.0146, time=0.793s
[batch B=20] epoch 57/60, MSE=0.0144, time=0.832s
[batch B=20] epoch 58/60, MSE=0.0138, time=0.808s
[batch B=20] epoch 59/60, MSE=0.0139, time=0.742s
[batch B=20] epoch 60/60, MSE=0.0136, time=0.748s
Train acc: 0.9957, Test acc: 0.9819
Train MSE: 0.0136, Test MSE: 0.0696
Avg epoch time: 0.7813s, Total train time: 46.88s
Inference time per sample: 1.576338e-06s
Misclassified per digit (test): {0: 9, 1: 8, 2: 21, 3: 19, 4: 13, 5: 21, 6: 19, 7: 22, 8: 23, 9: 26}
```

```
=== CONFIG: case=1, hidden=800, eta=0.01 (B=20) ===
[batch B=20] epoch 1/60, MSE=0.4079, time=1.786s
[batch B=20] epoch 2/60, MSE=0.3575, time=1.748s
[batch B=20] epoch 3/60, MSE=0.3313, time=1.789s
[batch B=20] epoch 4/60, MSE=0.3193, time=1.756s
[batch B=20] epoch 5/60, MSE=0.3066, time=1.755s
[batch B=20] epoch 6/60, MSE=0.2861, time=1.766s
[batch B=20] epoch 7/60, MSE=0.2775, time=1.800s
[batch B=20] epoch 8/60, MSE=0.2572, time=1.770s
[batch B=20] epoch 9/60, MSE=0.2468, time=1.780s
[batch B=20] epoch 10/60, MSE=0.2294, time=1.740s
[batch B=20] epoch 11/60, MSE=0.2183, time=1.777s
[batch B=20] epoch 12/60, MSE=0.2083, time=1.767s
[batch B=20] epoch 13/60, MSE=0.2009, time=1.767s
[batch B=20] epoch 14/60, MSE=0.1905, time=1.798s
[batch B=20] epoch 15/60, MSE=0.1826, time=1.774s
[batch B=20] epoch 16/60, MSE=0.1752, time=1.805s
[batch B=20] epoch 17/60, MSE=0.1679, time=1.807s
[batch B=20] epoch 18/60, MSE=0.1582, time=1.792s
[batch B=20] epoch 19/60, MSE=0.1518, time=1.781s
[batch B=20] epoch 20/60, MSE=0.1453, time=1.782s
[batch B=20] epoch 21/60, MSE=0.1429, time=1.793s
[batch B=20] epoch 22/60, MSE=0.1357, time=1.788s
[batch B=20] epoch 23/60, MSE=0.1317, time=1.807s
```

```
[batch B=20] epoch 24/60, MSE=0.1293, time=1.814s
[batch B=20] epoch 25/60, MSE=0.1203, time=1.732s
[batch B=20] epoch 26/60, MSE=0.1174, time=1.744s
[batch B=20] epoch 27/60, MSE=0.1142, time=1.848s
[batch B=20] epoch 28/60, MSE=0.1110, time=1.779s
[batch B=20] epoch 29/60, MSE=0.1068, time=1.800s
[batch B=20] epoch 30/60, MSE=0.1042, time=1.777s
[batch B=20] epoch 31/60, MSE=0.1010, time=1.769s
[batch B=20] epoch 32/60, MSE=0.0991, time=1.789s
[batch B=20] epoch 33/60, MSE=0.0945, time=1.776s
[batch B=20] epoch 34/60, MSE=0.0920, time=1.758s
[batch B=20] epoch 35/60, MSE=0.0889, time=1.808s
[batch B=20] epoch 36/60, MSE=0.0898, time=1.776s
[batch B=20] epoch 37/60, MSE=0.0860, time=1.771s
[batch B=20] epoch 38/60, MSE=0.0819, time=1.771s
[batch B=20] epoch 39/60, MSE=0.0835, time=1.787s
[batch B=20] epoch 40/60, MSE=0.0802, time=1.776s
[batch B=20] epoch 41/60, MSE=0.0771, time=1.761s
[batch B=20] epoch 42/60, MSE=0.0751, time=1.785s
[batch B=20] epoch 43/60, MSE=0.0745, time=1.754s
[batch B=20] epoch 44/60, MSE=0.0729, time=1.777s
[batch B=20] epoch 45/60, MSE=0.0694, time=1.770s
[batch B=20] epoch 46/60, MSE=0.0689, time=1.764s
[batch B=20] epoch 47/60, MSE=0.0667, time=1.774s
[batch B=20] epoch 48/60, MSE=0.0647, time=1.761s
[batch B=20] epoch 49/60, MSE=0.0648, time=1.769s
[batch B=20] epoch 50/60, MSE=0.0672, time=1.768s
[batch B=20] epoch 51/60, MSE=0.0612, time=1.762s
[batch B=20] epoch 52/60, MSE=0.0615, time=1.742s
[batch B=20] epoch 53/60, MSE=0.0591, time=1.778s
[batch B=20] epoch 54/60, MSE=0.0571, time=1.783s
[batch B=20] epoch 55/60, MSE=0.0564, time=1.750s
[batch B=20] epoch 56/60, MSE=0.0553, time=1.744s
[batch B=20] epoch 57/60, MSE=0.0545, time=1.783s
[batch B=20] epoch 58/60, MSE=0.0529, time=1.731s
[batch B=20] epoch 59/60, MSE=0.0533, time=1.713s
[batch B=20] epoch 60/60, MSE=0.0519, time=1.726s
Train acc: 0.9872, Test acc: 0.9764
Train MSE: 0.0519, Test MSE: 0.0887
Avg epoch time: 1.7733s, Total train time: 106.40s
Inference time per sample: 2.991982e-06s
```

Misclassified per digit (test): {0: 8, 1: 16, 2: 22, 3: 21, 4: 20, 5: 24, 6: 22, 7: 39, 8: 26, 9: 38}

```
=== CONFIG: case=1, hidden=800, eta=0.03 (B=20) ===  
[batch B=20] epoch 1/60, MSE=0.3489, time=1.763s  
[batch B=20] epoch 2/60, MSE=0.3054, time=1.786s  
[batch B=20] epoch 3/60, MSE=0.2601, time=1.700s  
[batch B=20] epoch 4/60, MSE=0.2283, time=1.733s  
[batch B=20] epoch 5/60, MSE=0.1985, time=1.759s  
[batch B=20] epoch 6/60, MSE=0.1763, time=1.745s  
[batch B=20] epoch 7/60, MSE=0.1563, time=1.731s  
[batch B=20] epoch 8/60, MSE=0.1386, time=1.783s  
[batch B=20] epoch 9/60, MSE=0.1236, time=1.710s  
[batch B=20] epoch 10/60, MSE=0.1182, time=1.737s  
[batch B=20] epoch 11/60, MSE=0.1041, time=1.724s  
[batch B=20] epoch 12/60, MSE=0.0987, time=1.777s  
[batch B=20] epoch 13/60, MSE=0.0913, time=1.717s  
[batch B=20] epoch 14/60, MSE=0.0869, time=1.725s  
[batch B=20] epoch 15/60, MSE=0.0795, time=1.763s  
[batch B=20] epoch 16/60, MSE=0.0836, time=1.715s  
[batch B=20] epoch 17/60, MSE=0.0698, time=1.748s  
[batch B=20] epoch 18/60, MSE=0.0670, time=1.774s  
[batch B=20] epoch 19/60, MSE=0.0640, time=1.867s  
[batch B=20] epoch 20/60, MSE=0.0589, time=1.728s  
[batch B=20] epoch 21/60, MSE=0.0542, time=1.714s  
[batch B=20] epoch 22/60, MSE=0.0534, time=1.771s  
[batch B=20] epoch 23/60, MSE=0.0510, time=1.734s  
[batch B=20] epoch 24/60, MSE=0.0465, time=1.771s  
[batch B=20] epoch 25/60, MSE=0.0468, time=1.732s  
[batch B=20] epoch 26/60, MSE=0.0428, time=1.714s  
[batch B=20] epoch 27/60, MSE=0.0406, time=1.735s  
[batch B=20] epoch 28/60, MSE=0.0394, time=1.699s  
[batch B=20] epoch 29/60, MSE=0.0386, time=1.789s  
[batch B=20] epoch 30/60, MSE=0.0353, time=1.705s  
[batch B=20] epoch 31/60, MSE=0.0351, time=1.708s  
[batch B=20] epoch 32/60, MSE=0.0328, time=1.773s  
[batch B=20] epoch 33/60, MSE=0.0331, time=1.718s  
[batch B=20] epoch 34/60, MSE=0.0302, time=1.780s  
[batch B=20] epoch 35/60, MSE=0.0293, time=1.721s  
[batch B=20] epoch 36/60, MSE=0.0305, time=1.786s  
[batch B=20] epoch 37/60, MSE=0.0271, time=1.736s  
[batch B=20] epoch 38/60, MSE=0.0273, time=1.723s
```



```

[batch B=20] epoch 39/60, MSE=0.0257, time=1.748s
[batch B=20] epoch 40/60, MSE=0.0244, time=1.783s
[batch B=20] epoch 41/60, MSE=0.0244, time=1.758s
[batch B=20] epoch 42/60, MSE=0.0233, time=1.687s
[batch B=20] epoch 43/60, MSE=0.0230, time=1.720s
[batch B=20] epoch 44/60, MSE=0.0213, time=1.751s
[batch B=20] epoch 45/60, MSE=0.0212, time=1.729s
[batch B=20] epoch 46/60, MSE=0.0205, time=1.773s
[batch B=20] epoch 47/60, MSE=0.0197, time=1.749s
[batch B=20] epoch 48/60, MSE=0.0194, time=1.706s
[batch B=20] epoch 49/60, MSE=0.0190, time=1.698s
[batch B=20] epoch 50/60, MSE=0.0191, time=1.760s
[batch B=20] epoch 51/60, MSE=0.0180, time=1.712s
[batch B=20] epoch 52/60, MSE=0.0181, time=1.770s
[batch B=20] epoch 53/60, MSE=0.0182, time=1.778s
[batch B=20] epoch 54/60, MSE=0.0174, time=1.730s
[batch B=20] epoch 55/60, MSE=0.0162, time=1.804s
[batch B=20] epoch 56/60, MSE=0.0164, time=1.774s
[batch B=20] epoch 57/60, MSE=0.0154, time=1.722s
[batch B=20] epoch 58/60, MSE=0.0155, time=1.775s
[batch B=20] epoch 59/60, MSE=0.0150, time=1.694s
[batch B=20] epoch 60/60, MSE=0.0148, time=1.703s
Train acc: 0.9954, Test acc: 0.9826
Train MSE: 0.0148, Test MSE: 0.0686
Avg epoch time: 1.7437s, Total train time: 104.62s
Inference time per sample: 3.002205e-06s
Misclassified per digit (test): {0: 7, 1: 8, 2: 17, 3: 13, 4: 19, 5: 26, 6: 15, 7: 26, 8: 17, 9: 26}

```

```

=== CONFIG: case=2, hidden=400, eta=0.01 (B=20) ===
[batch B=20] epoch 1/60, MSE=0.3662, time=0.820s
[batch B=20] epoch 2/60, MSE=0.2739, time=0.810s
[batch B=20] epoch 3/60, MSE=0.2270, time=0.776s
[batch B=20] epoch 4/60, MSE=0.1995, time=0.768s
[batch B=20] epoch 5/60, MSE=0.1743, time=0.778s
[batch B=20] epoch 6/60, MSE=0.1582, time=0.782s
[batch B=20] epoch 7/60, MSE=0.1439, time=0.811s
[batch B=20] epoch 8/60, MSE=0.1312, time=0.796s
[batch B=20] epoch 9/60, MSE=0.1212, time=0.780s
[batch B=20] epoch 10/60, MSE=0.1132, time=0.806s
[batch B=20] epoch 11/60, MSE=0.1051, time=0.799s
[batch B=20] epoch 12/60, MSE=0.0985, time=0.821s

```

```
[batch B=20] epoch 13/60, MSE=0.0969, time=0.817s
[batch B=20] epoch 14/60, MSE=0.0898, time=0.791s
[batch B=20] epoch 15/60, MSE=0.0841, time=0.769s
[batch B=20] epoch 16/60, MSE=0.0801, time=0.679s
[batch B=20] epoch 17/60, MSE=0.0763, time=0.728s
[batch B=20] epoch 18/60, MSE=0.0737, time=0.629s
[batch B=20] epoch 19/60, MSE=0.0703, time=0.812s
[batch B=20] epoch 20/60, MSE=0.0664, time=0.783s
[batch B=20] epoch 21/60, MSE=0.0645, time=0.805s
[batch B=20] epoch 22/60, MSE=0.0617, time=0.795s
[batch B=20] epoch 23/60, MSE=0.0597, time=0.757s
[batch B=20] epoch 24/60, MSE=0.0568, time=0.784s
[batch B=20] epoch 25/60, MSE=0.0549, time=0.742s
[batch B=20] epoch 26/60, MSE=0.0537, time=0.772s
[batch B=20] epoch 27/60, MSE=0.0518, time=0.793s
[batch B=20] epoch 28/60, MSE=0.0504, time=0.787s
[batch B=20] epoch 29/60, MSE=0.0480, time=0.752s
[batch B=20] epoch 30/60, MSE=0.0470, time=0.804s
[batch B=20] epoch 31/60, MSE=0.0454, time=0.667s
[batch B=20] epoch 32/60, MSE=0.0451, time=0.786s
[batch B=20] epoch 33/60, MSE=0.0426, time=0.795s
[batch B=20] epoch 34/60, MSE=0.0410, time=0.798s
[batch B=20] epoch 35/60, MSE=0.0398, time=0.782s
[batch B=20] epoch 36/60, MSE=0.0390, time=0.804s
[batch B=20] epoch 37/60, MSE=0.0382, time=0.811s
[batch B=20] epoch 38/60, MSE=0.0385, time=0.775s
[batch B=20] epoch 39/60, MSE=0.0369, time=0.813s
[batch B=20] epoch 40/60, MSE=0.0351, time=0.790s
[batch B=20] epoch 41/60, MSE=0.0342, time=0.684s
[batch B=20] epoch 42/60, MSE=0.0332, time=0.674s
[batch B=20] epoch 43/60, MSE=0.0327, time=0.681s
[batch B=20] epoch 44/60, MSE=0.0318, time=0.768s
[batch B=20] epoch 45/60, MSE=0.0314, time=0.777s
[batch B=20] epoch 46/60, MSE=0.0303, time=0.690s
[batch B=20] epoch 47/60, MSE=0.0295, time=0.806s
[batch B=20] epoch 48/60, MSE=0.0290, time=0.750s
[batch B=20] epoch 49/60, MSE=0.0283, time=0.671s
[batch B=20] epoch 50/60, MSE=0.0275, time=0.794s
[batch B=20] epoch 51/60, MSE=0.0269, time=0.793s
[batch B=20] epoch 52/60, MSE=0.0265, time=0.789s
[batch B=20] epoch 53/60, MSE=0.0257, time=0.783s
[batch B=20] epoch 54/60, MSE=0.0253, time=0.768s
[batch B=20] epoch 55/60, MSE=0.0250, time=0.811s
```



```
[batch B=20] epoch 56/60, MSE=0.0247, time=0.753s
[batch B=20] epoch 57/60, MSE=0.0237, time=0.791s
[batch B=20] epoch 58/60, MSE=0.0232, time=0.767s
[batch B=20] epoch 59/60, MSE=0.0229, time=0.807s
[batch B=20] epoch 60/60, MSE=0.0225, time=0.799s
Train acc: 0.9946, Test acc: 0.9831
Train MSE: 0.0225, Test MSE: 0.0668
Avg epoch time: 0.7720s, Total train time: 46.32s
Inference time per sample: 1.373563e-06s
Misclassified per digit (test): {0: 7, 1: 10, 2: 18, 3: 19, 4: 15, 5: 22, 6: 12, 7: 23, 8: 21, 9: 22}
```

```
=== CONFIG: case=2, hidden=400, eta=0.03 (B=20) ===
[batch B=20] epoch 1/60, MSE=0.2327, time=0.683s
[batch B=20] epoch 2/60, MSE=0.1622, time=0.646s
[batch B=20] epoch 3/60, MSE=0.1283, time=0.791s
[batch B=20] epoch 4/60, MSE=0.1080, time=0.668s
[batch B=20] epoch 5/60, MSE=0.0930, time=0.793s
[batch B=20] epoch 6/60, MSE=0.0821, time=0.794s
[batch B=20] epoch 7/60, MSE=0.0692, time=0.760s
[batch B=20] epoch 8/60, MSE=0.0627, time=0.765s
[batch B=20] epoch 9/60, MSE=0.0561, time=0.802s
[batch B=20] epoch 10/60, MSE=0.0511, time=0.759s
[batch B=20] epoch 11/60, MSE=0.0494, time=0.788s
[batch B=20] epoch 12/60, MSE=0.0421, time=0.735s
[batch B=20] epoch 13/60, MSE=0.0413, time=0.804s
[batch B=20] epoch 14/60, MSE=0.0398, time=0.759s
[batch B=20] epoch 15/60, MSE=0.0355, time=0.711s
[batch B=20] epoch 16/60, MSE=0.0321, time=0.762s
[batch B=20] epoch 17/60, MSE=0.0295, time=0.803s
[batch B=20] epoch 18/60, MSE=0.0296, time=0.788s
[batch B=20] epoch 19/60, MSE=0.0268, time=0.690s
[batch B=20] epoch 20/60, MSE=0.0248, time=0.805s
[batch B=20] epoch 21/60, MSE=0.0231, time=0.746s
[batch B=20] epoch 22/60, MSE=0.0217, time=0.810s
[batch B=20] epoch 23/60, MSE=0.0207, time=0.798s
[batch B=20] epoch 24/60, MSE=0.0199, time=0.792s
[batch B=20] epoch 25/60, MSE=0.0189, time=0.756s
[batch B=20] epoch 26/60, MSE=0.0183, time=0.793s
[batch B=20] epoch 27/60, MSE=0.0170, time=0.657s
[batch B=20] epoch 28/60, MSE=0.0168, time=0.772s
[batch B=20] epoch 29/60, MSE=0.0160, time=0.784s
```

```

[batch B=20] epoch 30/60, MSE=0.0152, time=0.672s
[batch B=20] epoch 31/60, MSE=0.0147, time=0.785s
[batch B=20] epoch 32/60, MSE=0.0142, time=0.812s
[batch B=20] epoch 33/60, MSE=0.0135, time=0.740s
[batch B=20] epoch 34/60, MSE=0.0132, time=0.767s
[batch B=20] epoch 35/60, MSE=0.0127, time=0.757s
[batch B=20] epoch 36/60, MSE=0.0123, time=0.780s
[batch B=20] epoch 37/60, MSE=0.0118, time=0.804s
[batch B=20] epoch 38/60, MSE=0.0116, time=0.677s
[batch B=20] epoch 39/60, MSE=0.0110, time=0.802s
[batch B=20] epoch 40/60, MSE=0.0108, time=0.854s
[batch B=20] epoch 41/60, MSE=0.0105, time=0.757s
[batch B=20] epoch 42/60, MSE=0.0103, time=0.693s
[batch B=20] epoch 43/60, MSE=0.0100, time=0.763s
[batch B=20] epoch 44/60, MSE=0.0100, time=0.749s
[batch B=20] epoch 45/60, MSE=0.0099, time=0.748s
[batch B=20] epoch 46/60, MSE=0.0095, time=0.674s
[batch B=20] epoch 47/60, MSE=0.0092, time=0.820s
[batch B=20] epoch 48/60, MSE=0.0091, time=0.763s
[batch B=20] epoch 49/60, MSE=0.0089, time=0.783s
[batch B=20] epoch 50/60, MSE=0.0089, time=0.765s
[batch B=20] epoch 51/60, MSE=0.0086, time=0.790s
[batch B=20] epoch 52/60, MSE=0.0085, time=0.785s
[batch B=20] epoch 53/60, MSE=0.0084, time=0.780s
[batch B=20] epoch 54/60, MSE=0.0082, time=0.752s
[batch B=20] epoch 55/60, MSE=0.0081, time=0.753s
[batch B=20] epoch 56/60, MSE=0.0080, time=0.806s
[batch B=20] epoch 57/60, MSE=0.0079, time=0.766s
[batch B=20] epoch 58/60, MSE=0.0077, time=0.674s
[batch B=20] epoch 59/60, MSE=0.0077, time=0.722s
[batch B=20] epoch 60/60, MSE=0.0076, time=0.731s
Train acc: 0.9973, Test acc: 0.9839
Train MSE: 0.0076, Test MSE: 0.0631
Avg epoch time: 0.7589s, Total train time: 45.54s
Inference time per sample: 1.395254e-06s
Misclassified per digit (test): {0: 6, 1: 11, 2: 14, 3: 18, 4: 14, 5: 22, 6: 14, 7: 19, 8: 18, 9:
25}

```

```

=== CONFIG: case=2, hidden=800, eta=0.01 (B=20) ===
[batch B=20] epoch 1/60, MSE=0.3509, time=1.696s
[batch B=20] epoch 2/60, MSE=0.2662, time=1.663s
[batch B=20] epoch 3/60, MSE=0.2227, time=1.697s

```

[batch B=20] epoch 4/60, MSE=0.1908, time=1.694s
[batch B=20] epoch 5/60, MSE=0.1694, time=1.671s
[batch B=20] epoch 6/60, MSE=0.1539, time=1.756s
[batch B=20] epoch 7/60, MSE=0.1368, time=1.781s
[batch B=20] epoch 8/60, MSE=0.1268, time=1.718s
[batch B=20] epoch 9/60, MSE=0.1157, time=1.714s
[batch B=20] epoch 10/60, MSE=0.1086, time=1.753s
[batch B=20] epoch 11/60, MSE=0.1000, time=1.714s
[batch B=20] epoch 12/60, MSE=0.0940, time=1.770s
[batch B=20] epoch 13/60, MSE=0.0892, time=1.703s
[batch B=20] epoch 14/60, MSE=0.0840, time=1.749s
[batch B=20] epoch 15/60, MSE=0.0794, time=1.697s
[batch B=20] epoch 16/60, MSE=0.0756, time=1.745s
[batch B=20] epoch 17/60, MSE=0.0719, time=1.789s
[batch B=20] epoch 18/60, MSE=0.0680, time=1.767s
[batch B=20] epoch 19/60, MSE=0.0662, time=1.803s
[batch B=20] epoch 20/60, MSE=0.0630, time=1.790s
[batch B=20] epoch 21/60, MSE=0.0591, time=1.747s
[batch B=20] epoch 22/60, MSE=0.0575, time=1.734s
[batch B=20] epoch 23/60, MSE=0.0559, time=2.089s
[batch B=20] epoch 24/60, MSE=0.0526, time=1.907s
[batch B=20] epoch 25/60, MSE=0.0512, time=1.847s
[batch B=20] epoch 26/60, MSE=0.0491, time=1.713s
[batch B=20] epoch 27/60, MSE=0.0472, time=1.699s
[batch B=20] epoch 28/60, MSE=0.0455, time=1.743s
[batch B=20] epoch 29/60, MSE=0.0439, time=1.760s
[batch B=20] epoch 30/60, MSE=0.0420, time=1.749s
[batch B=20] epoch 31/60, MSE=0.0414, time=1.904s
[batch B=20] epoch 32/60, MSE=0.0397, time=1.746s
[batch B=20] epoch 33/60, MSE=0.0386, time=1.701s
[batch B=20] epoch 34/60, MSE=0.0377, time=1.693s
[batch B=20] epoch 35/60, MSE=0.0363, time=1.706s
[batch B=20] epoch 36/60, MSE=0.0353, time=1.699s
[batch B=20] epoch 37/60, MSE=0.0339, time=1.702s
[batch B=20] epoch 38/60, MSE=0.0333, time=1.733s
[batch B=20] epoch 39/60, MSE=0.0325, time=1.756s
[batch B=20] epoch 40/60, MSE=0.0312, time=1.737s
[batch B=20] epoch 41/60, MSE=0.0307, time=1.707s
[batch B=20] epoch 42/60, MSE=0.0296, time=1.699s
[batch B=20] epoch 43/60, MSE=0.0288, time=1.736s
[batch B=20] epoch 44/60, MSE=0.0283, time=1.862s
[batch B=20] epoch 45/60, MSE=0.0273, time=1.706s
[batch B=20] epoch 46/60, MSE=0.0268, time=1.779s

```
[batch B=20] epoch 47/60, MSE=0.0262, time=1.776s
[batch B=20] epoch 48/60, MSE=0.0254, time=1.693s
[batch B=20] epoch 49/60, MSE=0.0249, time=1.662s
[batch B=20] epoch 50/60, MSE=0.0242, time=1.718s
[batch B=20] epoch 51/60, MSE=0.0243, time=1.694s
[batch B=20] epoch 52/60, MSE=0.0231, time=1.673s
[batch B=20] epoch 53/60, MSE=0.0229, time=1.700s
[batch B=20] epoch 54/60, MSE=0.0227, time=1.703s
[batch B=20] epoch 55/60, MSE=0.0218, time=1.701s
[batch B=20] epoch 56/60, MSE=0.0213, time=1.673s
[batch B=20] epoch 57/60, MSE=0.0211, time=1.676s
[batch B=20] epoch 58/60, MSE=0.0204, time=1.749s
[batch B=20] epoch 59/60, MSE=0.0200, time=1.697s
[batch B=20] epoch 60/60, MSE=0.0197, time=1.673s
Train acc: 0.9953, Test acc: 0.9832
Train MSE: 0.0197, Test MSE: 0.0644
Avg epoch time: 1.7385s, Total train time: 104.31s
Inference time per sample: 2.646561e-06s
Misclassified per digit (test): {0: 7, 1: 9, 2: 18, 3: 14, 4: 16, 5: 24, 6: 13, 7: 23, 8: 23, 9:
21}
```

```
=== CONFIG: case=2, hidden=800, eta=0.03 (B=20) ===
[batch B=20] epoch 1/60, MSE=0.2209, time=1.712s
[batch B=20] epoch 2/60, MSE=0.1633, time=1.725s
[batch B=20] epoch 3/60, MSE=0.1194, time=1.687s
[batch B=20] epoch 4/60, MSE=0.0984, time=1.701s
[batch B=20] epoch 5/60, MSE=0.0848, time=1.750s
[batch B=20] epoch 6/60, MSE=0.0733, time=1.690s
[batch B=20] epoch 7/60, MSE=0.0655, time=1.686s
[batch B=20] epoch 8/60, MSE=0.0575, time=1.667s
[batch B=20] epoch 9/60, MSE=0.0526, time=1.737s
[batch B=20] epoch 10/60, MSE=0.0490, time=1.652s
[batch B=20] epoch 11/60, MSE=0.0426, time=1.736s
[batch B=20] epoch 12/60, MSE=0.0393, time=1.662s
[batch B=20] epoch 13/60, MSE=0.0383, time=1.670s
[batch B=20] epoch 14/60, MSE=0.0333, time=1.643s
[batch B=20] epoch 15/60, MSE=0.0306, time=1.730s
[batch B=20] epoch 16/60, MSE=0.0285, time=1.715s
[batch B=20] epoch 17/60, MSE=0.0267, time=1.749s
[batch B=20] epoch 18/60, MSE=0.0286, time=1.703s
[batch B=20] epoch 19/60, MSE=0.0226, time=1.697s
[batch B=20] epoch 20/60, MSE=0.0222, time=1.750s
```

```
[batch B=20] epoch 21/60, MSE=0.0200, time=1.671s
[batch B=20] epoch 22/60, MSE=0.0202, time=1.717s
[batch B=20] epoch 23/60, MSE=0.0181, time=1.714s
[batch B=20] epoch 24/60, MSE=0.0174, time=1.715s
[batch B=20] epoch 25/60, MSE=0.0159, time=1.659s
[batch B=20] epoch 26/60, MSE=0.0150, time=1.741s
[batch B=20] epoch 27/60, MSE=0.0143, time=1.658s
[batch B=20] epoch 28/60, MSE=0.0139, time=1.683s
[batch B=20] epoch 29/60, MSE=0.0131, time=1.694s
[batch B=20] epoch 30/60, MSE=0.0126, time=1.737s
[batch B=20] epoch 31/60, MSE=0.0121, time=1.701s
[batch B=20] epoch 32/60, MSE=0.0120, time=1.712s
[batch B=20] epoch 33/60, MSE=0.0115, time=1.683s
[batch B=20] epoch 34/60, MSE=0.0113, time=1.673s
[batch B=20] epoch 35/60, MSE=0.0108, time=1.739s
[batch B=20] epoch 36/60, MSE=0.0103, time=1.680s
[batch B=20] epoch 37/60, MSE=0.0099, time=1.688s
[batch B=20] epoch 38/60, MSE=0.0096, time=1.692s
[batch B=20] epoch 39/60, MSE=0.0094, time=1.743s
[batch B=20] epoch 40/60, MSE=0.0093, time=1.747s
[batch B=20] epoch 41/60, MSE=0.0089, time=1.691s
[batch B=20] epoch 42/60, MSE=0.0087, time=1.707s
[batch B=20] epoch 43/60, MSE=0.0084, time=1.678s
[batch B=20] epoch 44/60, MSE=0.0083, time=1.736s
[batch B=20] epoch 45/60, MSE=0.0081, time=1.672s
[batch B=20] epoch 46/60, MSE=0.0080, time=1.752s
[batch B=20] epoch 47/60, MSE=0.0077, time=1.692s
[batch B=20] epoch 48/60, MSE=0.0076, time=1.679s
[batch B=20] epoch 49/60, MSE=0.0075, time=1.721s
[batch B=20] epoch 50/60, MSE=0.0073, time=1.672s
[batch B=20] epoch 51/60, MSE=0.0073, time=1.697s
[batch B=20] epoch 52/60, MSE=0.0071, time=1.743s
[batch B=20] epoch 53/60, MSE=0.0070, time=1.677s
[batch B=20] epoch 54/60, MSE=0.0069, time=1.692s
[batch B=20] epoch 55/60, MSE=0.0067, time=1.728s
[batch B=20] epoch 56/60, MSE=0.0066, time=1.706s
[batch B=20] epoch 57/60, MSE=0.0065, time=1.670s
[batch B=20] epoch 58/60, MSE=0.0065, time=1.740s
[batch B=20] epoch 59/60, MSE=0.0063, time=1.705s
[batch B=20] epoch 60/60, MSE=0.0063, time=1.694s
Train acc: 0.9978, Test acc: 0.9842
Train MSE: 0.0063, Test MSE: 0.0591
Avg epoch time: 1.7027s, Total train time: 102.16s
```

Inference time per sample: 2.611184e-06s

Misclassified per digit (test): {0: 6, 1: 8, 2: 17, 3: 15, 4: 15, 5: 18, 6: 13, 7: 23, 8: 22, 9: 21}

=== BEST CONFIGURATION (BY TEST ACC) ===

case = 2, hidden = 800, eta = 0.03, test_acc = 0.9842

=== BATCH LEARNING: B=20, case=2, hidden=800, eta=0.03 ===

[batch B=20] epoch 1/60, MSE=0.2211, time=1.726s
[batch B=20] epoch 2/60, MSE=0.1536, time=1.635s
[batch B=20] epoch 3/60, MSE=0.1215, time=1.633s
[batch B=20] epoch 4/60, MSE=0.1024, time=1.652s
[batch B=20] epoch 5/60, MSE=0.0845, time=1.682s
[batch B=20] epoch 6/60, MSE=0.0745, time=1.720s
[batch B=20] epoch 7/60, MSE=0.0658, time=1.710s
[batch B=20] epoch 8/60, MSE=0.0575, time=1.668s
[batch B=20] epoch 9/60, MSE=0.0517, time=1.754s
[batch B=20] epoch 10/60, MSE=0.0471, time=1.693s
[batch B=20] epoch 11/60, MSE=0.0443, time=1.731s
[batch B=20] epoch 12/60, MSE=0.0395, time=1.739s
[batch B=20] epoch 13/60, MSE=0.0359, time=1.844s
[batch B=20] epoch 14/60, MSE=0.0336, time=1.751s
[batch B=20] epoch 15/60, MSE=0.0306, time=1.755s
[batch B=20] epoch 16/60, MSE=0.0276, time=1.645s
[batch B=20] epoch 17/60, MSE=0.0270, time=1.672s
[batch B=20] epoch 18/60, MSE=0.0241, time=1.703s
[batch B=20] epoch 19/60, MSE=0.0231, time=1.695s
[batch B=20] epoch 20/60, MSE=0.0212, time=1.679s
[batch B=20] epoch 21/60, MSE=0.0201, time=1.716s
[batch B=20] epoch 22/60, MSE=0.0187, time=1.730s
[batch B=20] epoch 23/60, MSE=0.0178, time=1.690s
[batch B=20] epoch 24/60, MSE=0.0167, time=1.709s
[batch B=20] epoch 25/60, MSE=0.0160, time=1.693s
[batch B=20] epoch 26/60, MSE=0.0149, time=1.675s
[batch B=20] epoch 27/60, MSE=0.0151, time=1.729s
[batch B=20] epoch 28/60, MSE=0.0140, time=1.730s
[batch B=20] epoch 29/60, MSE=0.0132, time=1.660s
[batch B=20] epoch 30/60, MSE=0.0128, time=1.733s
[batch B=20] epoch 31/60, MSE=0.0124, time=1.667s
[batch B=20] epoch 32/60, MSE=0.0122, time=1.653s
[batch B=20] epoch 33/60, MSE=0.0120, time=1.731s
[batch B=20] epoch 34/60, MSE=0.0109, time=1.652s


```

[batch B=20] epoch 35/60, MSE=0.0105, time=1.630s
[batch B=20] epoch 36/60, MSE=0.0102, time=1.676s
[batch B=20] epoch 37/60, MSE=0.0101, time=1.660s
[batch B=20] epoch 38/60, MSE=0.0097, time=1.672s
[batch B=20] epoch 39/60, MSE=0.0093, time=1.648s
[batch B=20] epoch 40/60, MSE=0.0091, time=1.700s
[batch B=20] epoch 41/60, MSE=0.0088, time=1.728s
[batch B=20] epoch 42/60, MSE=0.0087, time=1.720s
[batch B=20] epoch 43/60, MSE=0.0085, time=1.682s
[batch B=20] epoch 44/60, MSE=0.0083, time=1.698s
[batch B=20] epoch 45/60, MSE=0.0080, time=1.677s
[batch B=20] epoch 46/60, MSE=0.0079, time=1.684s
[batch B=20] epoch 47/60, MSE=0.0077, time=1.718s
[batch B=20] epoch 48/60, MSE=0.0077, time=1.706s
[batch B=20] epoch 49/60, MSE=0.0074, time=1.682s
[batch B=20] epoch 50/60, MSE=0.0073, time=1.639s
[batch B=20] epoch 51/60, MSE=0.0072, time=1.726s
[batch B=20] epoch 52/60, MSE=0.0072, time=1.697s
[batch B=20] epoch 53/60, MSE=0.0069, time=1.695s
[batch B=20] epoch 54/60, MSE=0.0069, time=1.715s
[batch B=20] epoch 55/60, MSE=0.0067, time=1.747s
[batch B=20] epoch 56/60, MSE=0.0066, time=1.631s
[batch B=20] epoch 57/60, MSE=0.0066, time=1.653s
[batch B=20] epoch 58/60, MSE=0.0064, time=1.657s
[batch B=20] epoch 59/60, MSE=0.0064, time=1.725s
[batch B=20] epoch 60/60, MSE=0.0063, time=1.681s
[B=20] Train acc: 0.9977, Test acc: 0.9849
[B=20] Train MSE: 0.0063, Test MSE: 0.0593
[B=20] Avg epoch time: 1.6950s, Total train time: 101.70s
[B=20] Inference time per sample: 2.615581e-06s
[B=20] Misclassified per digit (test): {0: 6, 1: 9, 2: 17, 3: 14, 4: 16, 5: 19, 6: 11, 7: 19, 8:
21, 9: 19}

```

```

=== BATCH LEARNING: B=200, case=2, hidden=800, eta=0.03 ===

```

```

[batch B=200] epoch 1/60, MSE=0.6488, time=0.402s
[batch B=200] epoch 2/60, MSE=0.4318, time=0.401s
[batch B=200] epoch 3/60, MSE=0.3611, time=0.413s
[batch B=200] epoch 4/60, MSE=0.3214, time=0.408s
[batch B=200] epoch 5/60, MSE=0.2948, time=0.428s
[batch B=200] epoch 6/60, MSE=0.2726, time=0.414s
[batch B=200] epoch 7/60, MSE=0.2563, time=0.412s
[batch B=200] epoch 8/60, MSE=0.2415, time=0.406s
[batch B=200] epoch 9/60, MSE=0.2292, time=0.422s

```

[batch B=200] epoch 10/60, MSE=0.2183, time=0.414s
[batch B=200] epoch 11/60, MSE=0.2079, time=0.417s
[batch B=200] epoch 12/60, MSE=0.1994, time=0.418s
[batch B=200] epoch 13/60, MSE=0.1912, time=0.415s
[batch B=200] epoch 14/60, MSE=0.1839, time=0.417s
[batch B=200] epoch 15/60, MSE=0.1776, time=0.416s
[batch B=200] epoch 16/60, MSE=0.1709, time=0.402s
[batch B=200] epoch 17/60, MSE=0.1651, time=0.418s
[batch B=200] epoch 18/60, MSE=0.1595, time=0.415s
[batch B=200] epoch 19/60, MSE=0.1547, time=0.423s
[batch B=200] epoch 20/60, MSE=0.1499, time=0.419s
[batch B=200] epoch 21/60, MSE=0.1456, time=0.421s
[batch B=200] epoch 22/60, MSE=0.1418, time=0.414s
[batch B=200] epoch 23/60, MSE=0.1372, time=0.418s
[batch B=200] epoch 24/60, MSE=0.1337, time=0.414s
[batch B=200] epoch 25/60, MSE=0.1301, time=0.416s
[batch B=200] epoch 26/60, MSE=0.1268, time=0.414s
[batch B=200] epoch 27/60, MSE=0.1236, time=0.418s
[batch B=200] epoch 28/60, MSE=0.1213, time=0.435s
[batch B=200] epoch 29/60, MSE=0.1178, time=0.419s
[batch B=200] epoch 30/60, MSE=0.1152, time=0.412s
[batch B=200] epoch 31/60, MSE=0.1128, time=0.411s
[batch B=200] epoch 32/60, MSE=0.1101, time=0.413s
[batch B=200] epoch 33/60, MSE=0.1078, time=0.413s
[batch B=200] epoch 34/60, MSE=0.1058, time=0.411s
[batch B=200] epoch 35/60, MSE=0.1034, time=0.412s
[batch B=200] epoch 36/60, MSE=0.1012, time=0.408s
[batch B=200] epoch 37/60, MSE=0.0997, time=0.414s
[batch B=200] epoch 38/60, MSE=0.0972, time=0.413s
[batch B=200] epoch 39/60, MSE=0.0954, time=0.415s
[batch B=200] epoch 40/60, MSE=0.0935, time=0.419s
[batch B=200] epoch 41/60, MSE=0.0923, time=0.418s
[batch B=200] epoch 42/60, MSE=0.0901, time=0.406s
[batch B=200] epoch 43/60, MSE=0.0885, time=0.419s
[batch B=200] epoch 44/60, MSE=0.0873, time=0.404s
[batch B=200] epoch 45/60, MSE=0.0857, time=0.418s
[batch B=200] epoch 46/60, MSE=0.0847, time=0.411s
[batch B=200] epoch 47/60, MSE=0.0827, time=0.413s
[batch B=200] epoch 48/60, MSE=0.0813, time=0.413s
[batch B=200] epoch 49/60, MSE=0.0801, time=0.417s
[batch B=200] epoch 50/60, MSE=0.0788, time=0.410s
[batch B=200] epoch 51/60, MSE=0.0780, time=0.421s
[batch B=200] epoch 52/60, MSE=0.0763, time=0.416s


```
[batch B=200] epoch 53/60, MSE=0.0752, time=0.415s
[batch B=200] epoch 54/60, MSE=0.0746, time=0.416s
[batch B=200] epoch 55/60, MSE=0.0729, time=0.409s
[batch B=200] epoch 56/60, MSE=0.0719, time=0.409s
[batch B=200] epoch 57/60, MSE=0.0707, time=0.414s
[batch B=200] epoch 58/60, MSE=0.0698, time=0.428s
[batch B=200] epoch 59/60, MSE=0.0688, time=0.423s
[batch B=200] epoch 60/60, MSE=0.0679, time=0.420s
[B=200] Train acc: 0.9843, Test acc: 0.9763
[B=200] Train MSE: 0.0679, Test MSE: 0.0884
[B=200] Avg epoch time: 0.4148s, Total train time: 24.89s
[B=200] Inference time per sample: 2.604380e-06s
[B=200] Misclassified per digit (test): {0: 10, 1: 13, 2: 27, 3: 20, 4: 21, 5: 27, 6: 20, 7: 3
4, 8: 27, 9: 38}
```